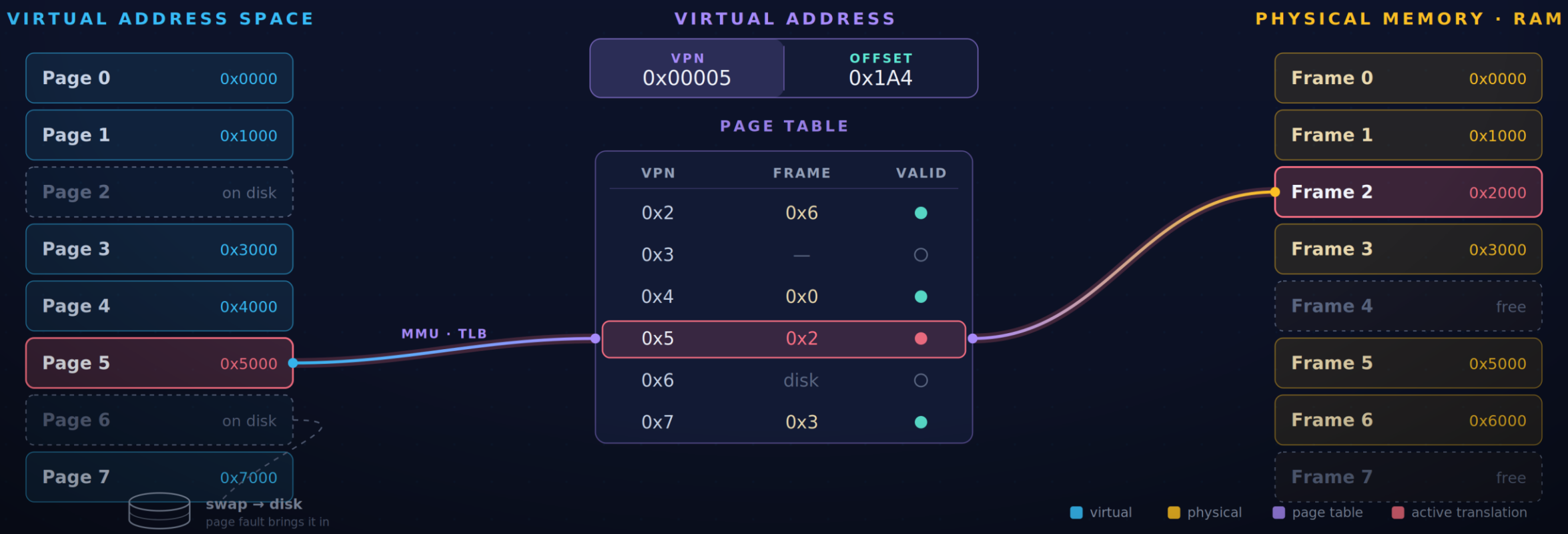
Memory Management

Adiesha Liyana Ralalage

CSCI 460 · OPERATING SYSTEMS

Memory Management

How the OS gives every process its own view of memory — and where that memory really lives.



Early memory management systems

- **Early computer systems supported only one user and one program at a time.**
  - This approach is known as **uniprogramming**.
- **To execute a program, the entire program had to be loaded contiguously into main memory.**
- **What are the implications?**
  - **Memory constraints and inefficiency**
    - The program size was **limited by the amount of available memory**.
    - If a program did not use all the allocated memory, the **unused memory was wasted**.
    - When the program was **waiting for an I/O operation**, the CPU remained **idle**.
    - As a result, **CPU and memory resources were often underutilized**.

- **In a multiprogramming system, the user portion of memory must be divided to accommodate multiple processes.**
- **The operating system dynamically manages this division of memory.**
  - This process is known as **memory management**.
- **Memory management determines how memory is allocated and used by different processes.**
- **We will examine several memory-management schemes that have been or are used in operating systems.**

- Some of the memory-management techniques we will study are relatively old and may not be used directly in modern systems.
- However, understanding these techniques is important because they provide the fundamental concepts behind modern memory-management systems.
- They also help us understand why modern operating systems use more sophisticated memory-management mechanisms.
- Although some of these techniques are old, the underlying ideas are still valuable.
- **Learning them can help us develop techniques and ways of thinking that can be applied to other problems.**

Memory management requirements

- Relocation
  - The ability to load a program into any available memory location and adjust its memory references dynamically.
- Protection
  - Ensuring that each program's memory space is isolated to prevent unauthorized access or interference from other programs, critical in multiprogramming to avoid crashes or data corruption.
- Sharing
  - Allowing multiple programs or users to access common memory areas (e.g., shared libraries or data) efficiently, while maintaining control to avoid conflicts, especially in multiprogramming environments.
- Logical organization
  - Structuring memory to support program modularity (e.g., separating code, data, and stack) for easier development and execution
- Physical organization
  - Managing the actual hardware memory (e.g., RAM, registers) to allocate and deallocate contiguous or non-contiguous blocks efficiently, addressing fragmentation challenges in early systems and optimizing resource use in modern ones.

Relocation

- In multiprogramming system, available memory is shared among multiple processes.
- Typically, programmers do not know in where the program will be in the main memory in advance.
- Active processes need to be able to be swapped in and out of main memory in order to maximize processor utilization.
- This requirement raises a lot of technical problems.
  - How can the programmer access memory without having to worry about relocation?

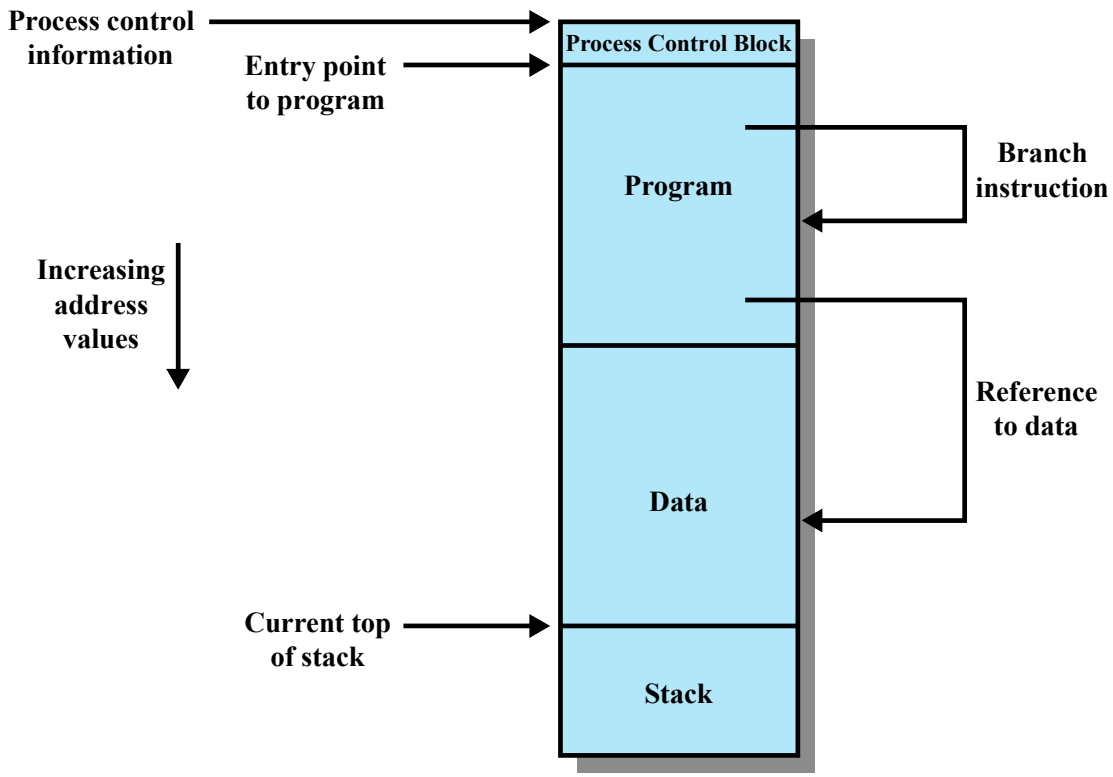


Figure 7.1 Addressing Requirements for a Process

Protection

- Each process needs to be protected against unwanted access by other programs.
- Relocation requirement makes harder to enforce protection.
  - Location of a program in main memory is unpredictable.
- Processes need to acquire permission to reference memory locations for reading and writing purposes.
- Memory references generated by process must be checked at run time.
- Schemes that support relocation must also support pretection.

Sharing

- Simultaneously, programs should be able to access some shared memory.
  - If there is a common library that is used by several programs, we should not the load the library more than once to conserve main memory.
- Memory management must allow controlled access to shared areas of memory without compromising protection.
- Mechanisms used to support relocation support sharing capabilities.

Logical organization

- Main memory and secondary memory are usually organized as a linear or one-dimensional address space..
- Programs are typically not like this.
- Programs have modules.
  - Some parts of programs are unmodifiable.
  - Some parts can be modified.
- OS needs effective deal with linearity of hardware and modularity of programs.

Physical organization

- Computer memory is organized in at least two levels: **main memory** (fast, volatile, limited in size) and **secondary memory** (slower, cheaper, much larger, non-volatile).
- Speed and cost trade off directly against each other: faster access means higher cost per byte — so fast memory is kept small and slow memory is made large.
- The OS can exploit this by keeping programs in secondary memory when they aren't executing or are blocked on long I/O waits, freeing main memory for active work.
- When the main memory available to a program is too small to hold it all at once, its code and data must be moved in and out during execution.
  - In early systems this was the programmer's job — a technique called **overlaying**, where different program sections are loaded into the same memory region as they're needed.
- Overlaying is impractical, and **relocation** makes it worse: the programmer can't know ahead of time where in main memory the program will be placed, or how much space will be available (especially with multiprogramming).
- Therefore, moving information between the two memory levels must be the **OS's responsibility, not the programmer’s.**

Partitioning schemes

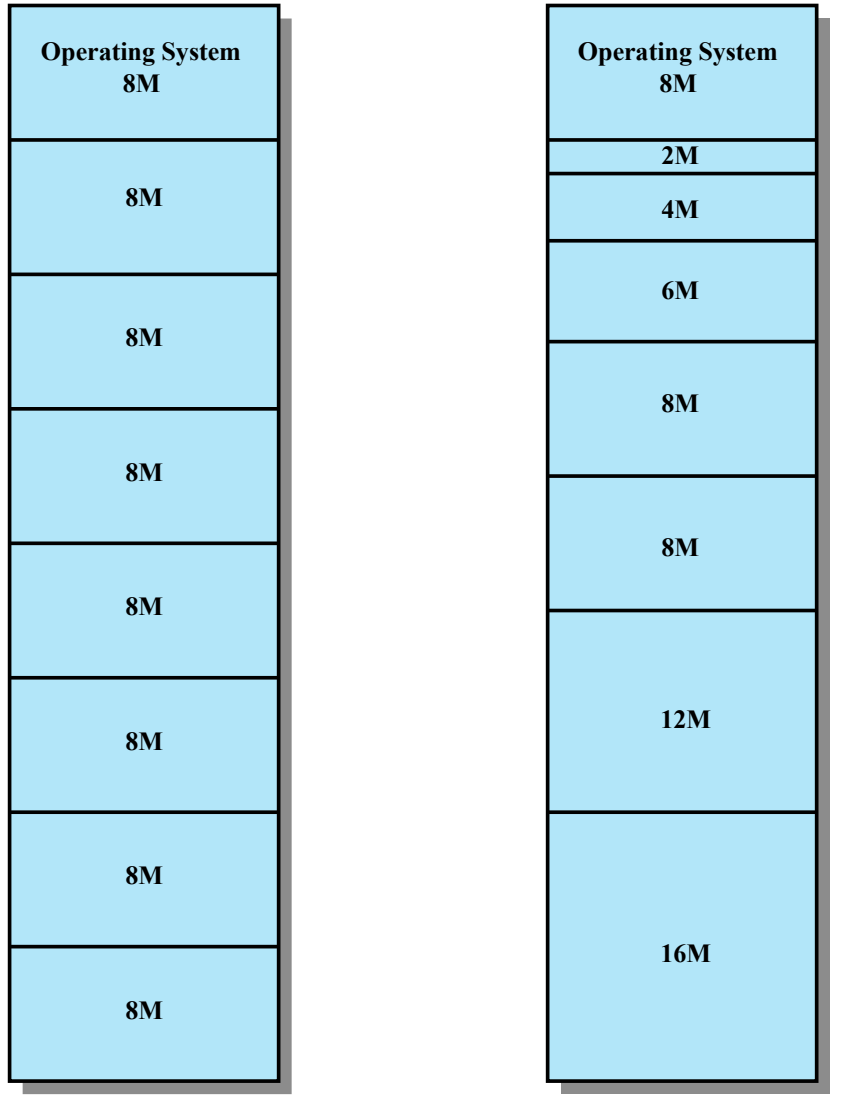
- Now, let’s look at several partitioning schemes that would allow us to manage memory.

Fixed partitioning

- First, assume that OS occupies portion of main memory, and the rest of the main memory is available for use by multiple processes.
- Fixed partitioning is very simple.
  - Idea: partition the memory into set of fixed partitions.
  - Two ways to partition the memory.
    - Equal sized partitions.
    - Variable sized partitions.

Fixed partitioning

- Fixed partitioning is very simple.
  - Idea: partition the memory into set of fixed partitions.
  - Two ways to partition the memory.
    - Equal sized partitions.
    - Variable sized partitions.
- Any process whose size is less than or equal to the partition size can be loaded into the available partition.
- If partitions are full, and process is ready or in running state, the OS can swap a process out of any partition and load another process.



(a) Equal-size partitions

(b) Unequal-size partitions

Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory

Fixed partitioning

- In equal sized partitioning, what are the implications?
  - Program could be larger than the partition.
    - Programmer must use overlays to handle this problem.
  - Regardless of the size of the program, the entire partition is used by a particular process.
  - If you have lot of small programs running, space is not utilized properly. **(Internal fragmentation)**
  - Internal fragmentation occurs when a process is smaller than the partition it's allocated to — the leftover space inside the partition is wasted and can't be used by any other process.

Fixed partitioning

- In unequal sized partitioning, what are the implications?
  - Larger programs could be handled.
  - Slightly better at handling smaller programs as we do have smaller partitions.
  - However, we can still have internal fragmentation.

Placement algorithms

- Before moving to other partitioning schemes, let’s talk a little bit about placement algorithms for fixed partitioning.
- For equal sized fixed partitioning, placement is very easy
  - As long as there is a free partition, you can load the program to it.
  - If there are no available partitions, swap a process already in the main memory to the secondary storage and load the new program to that partition.
- For unequal sized partitioning, you can assign the new process to the smallest available partition.

Placement algorithms

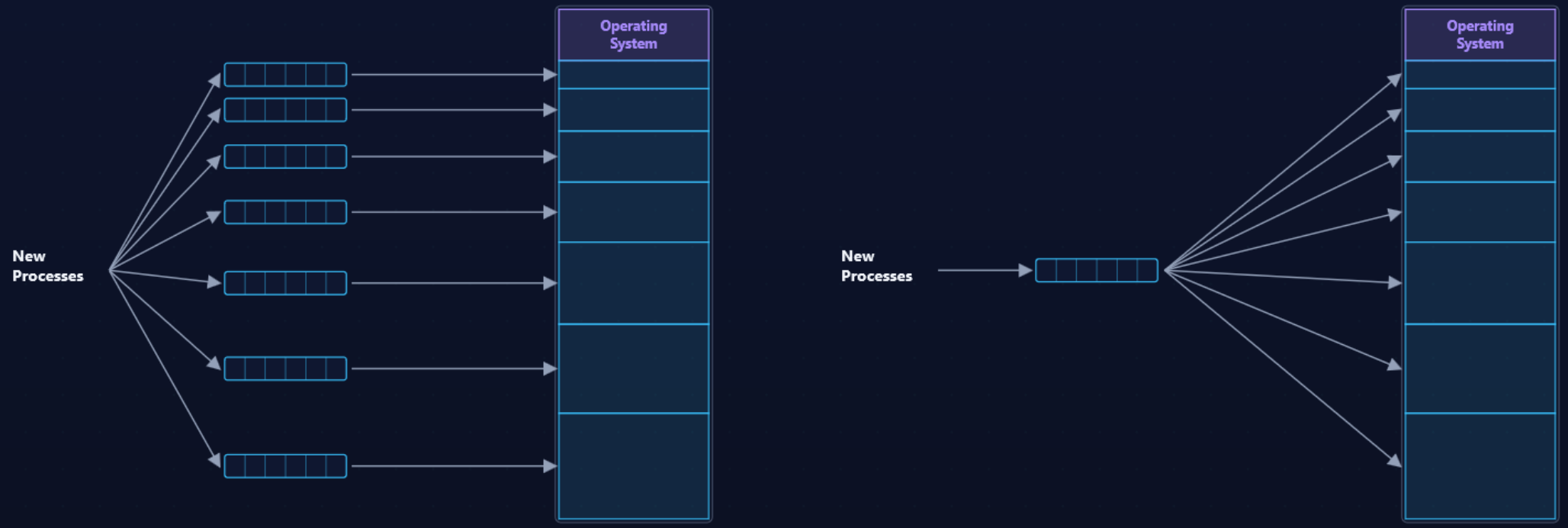
- For unequal sized partitioning, you can assign the new process to the smallest available partition.
  - We can use a scheduling queues for each partition, where each corresponding process will wait in the queue for the partition.
  - This is efficient in terms of low fragmentation as we always put the process into the best possible partition that minimizes the internal fragmentation, but this is not optimum from the point of view of the system.
  - Let's think about this using an example.
  - Disadvantages?
    - Wait time might be higher.
    - May need to swap processes to free up space.

- Suppose main memory is divided into four unequal partitions:
  - Partition 1 → 100 KB
  - Partition 2 → 200 KB
  - Partition 3 → 300 KB
  - Partition 4 → 500 KB
- Now suppose the following processes arrive:
  - P1 → 90 KB
  - P2 → 180 KB
  - P3 → 250 KB
  - P4 → 450 KB
- **P5 needs 150 KB**.

FIXED PARTITIONING · SCHEDULING

Memory Assignment

Two ways to schedule processes into unequal fixed partitions.



(a) One process queue per partition

Each partition has its own queue — a partition can sit idle while other queues are backed up.

(b) Single queue

One queue for all — each freed partition takes the best-fitting waiting process (better utilization).

Memory assignment for fixed partitioning.

Placement algorithms

- We can also have one queue for all the processes.
  - Then, we will allocate the processes to the best available smallest partition in the main memory.
  - This improves the overall wait time.
  - Obviously, if all the partitions are being used, we need to swap processes to free up space.

Fixed partitioning

- Overall, there are several disadvantages in fixed partitioning, even though it makes memory management somewhat easy.
  - A program may be too large for the available partition, then overlays must be used.
  - Internal fragmentation.
  - The number of partitions specified at system generation time limits the number of active (not suspended) processes in the system.

Dynamic partitioning

- To overcome some problems in fixed partitioning, dynamic partitioning was introduced.
- Partitions are of variable length and number.
  - When a process in brought to memory, it is allocated exactly as much memory it requires.
- OS needs to keep track of boundaries of the partitions.

Dynamic Partitioning

64 MB memory — loading & swapping create scattered holes

a

MAIN MEMORY

DISK · swapped out

Free memory

total: 56 M

holes: 56 M

Initial

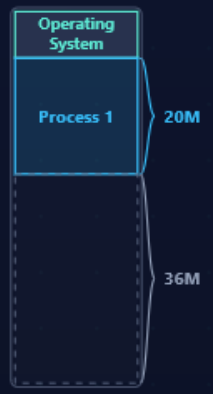
OS in memory; 56 MB free.

DYNAMIC PARTITIONING · EXTERNAL FRAGMENTATION

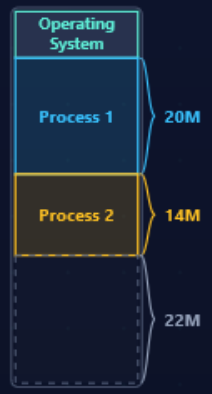
The Effect of Dynamic Partitioning

64 MB memory: loading and swapping leave a growing scatter of small holes.

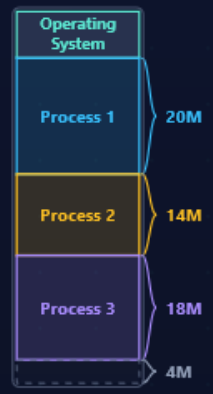
(a)
initial



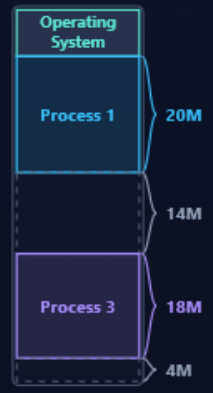
(b)
load P1



(c)
load P2



(d)
load P3 — hole too small for P4



(e)
swap out P2

(f)
load P4

(g)
swap out P1

(h)
swap in P2

16M free — but scattered (external fragmentation)

As processes load and swap, free memory external to the partitions becomes increasingly fragmented.

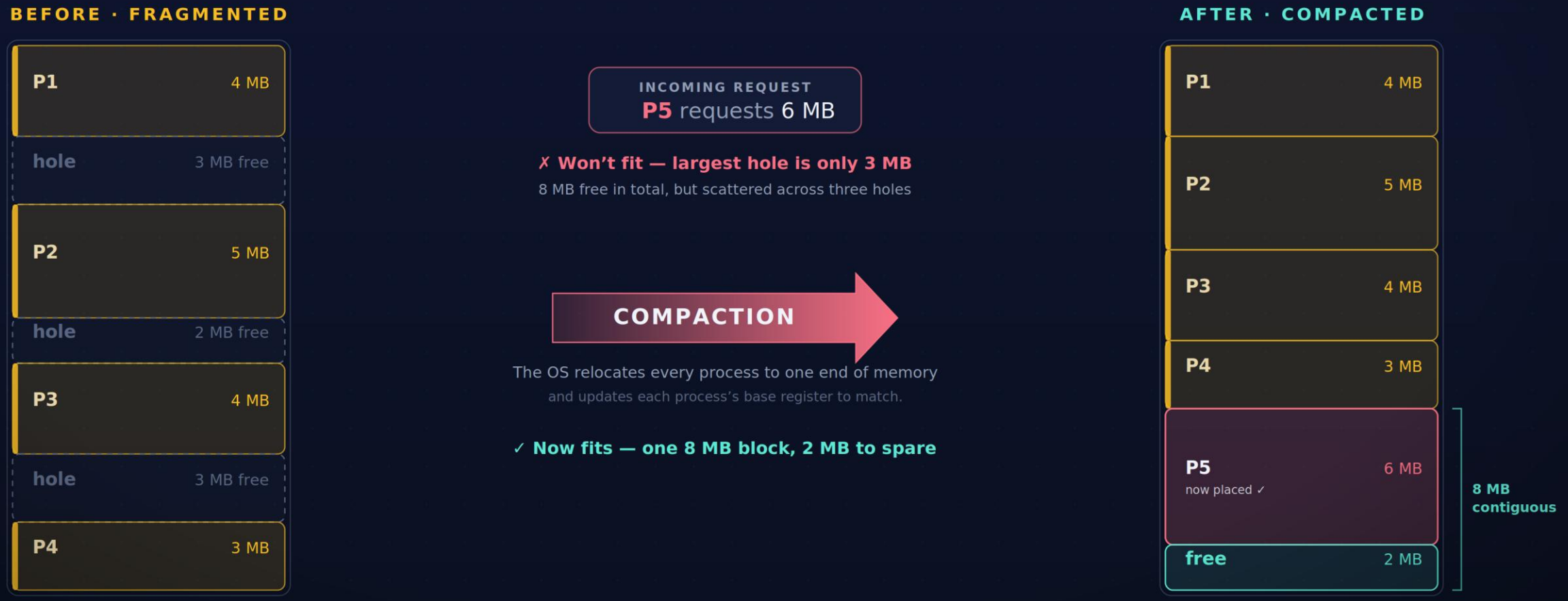
Dynamic Partition

- What do you think happens when we use this strategy?
  - You would see lot of small holes in main memory which cannot be occupied by processes.
  - This is called **External Fragmentation.**
- What can we do to remove this?
  - We can shift the processes into contiguous block.
  - This is called **compaction.**

DYNAMIC PARTITIONING

Compaction

Slide the allocated processes together so scattered free holes merge into one usable block.

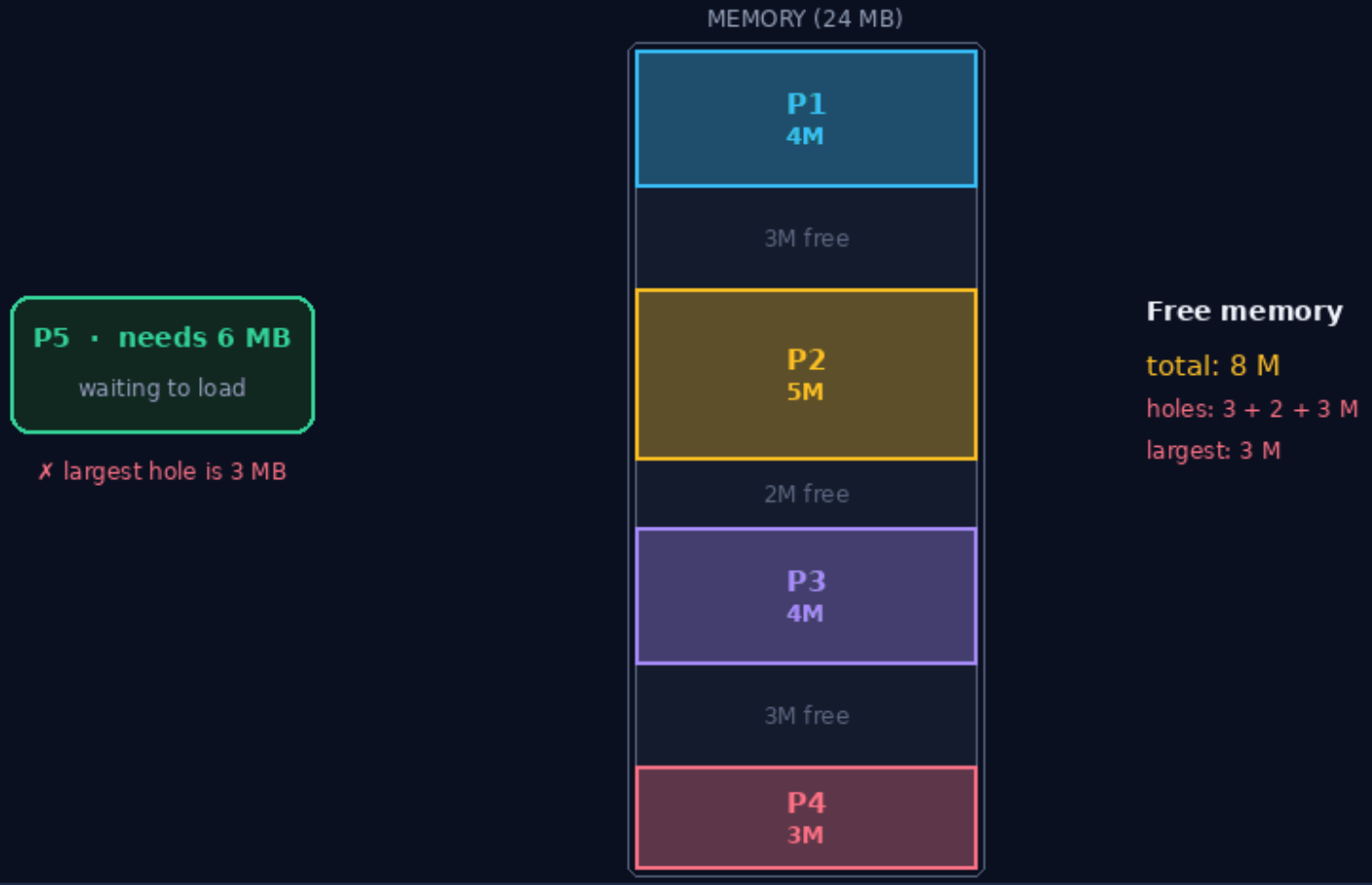


- Trade-off: compaction is expensive — it stalls execution while data is copied, and it requires dynamic relocation (processes movable at run time).

Compaction

Slide processes together to merge scattered free holes

before



Fragmented

8 MB free across three holes — largest is 3 MB, so P5 (6 MB) can't fit.

Placement algorithms?

- Since compaction is time consuming and resource hungry operation, OS needs to be clever in choosing the best position for a process to reside in dynamic partitioning.
- For example, if there are more than one free block, which one should OS use for placing the new process?

Placement algorithms for Dynamic Partitioning

Best-fit	First-fit	Next-fit
• Chooses the block that is closest in size to the request	• Begins to scan memory from the beginning and chooses the first available block that is large enough	• Begins to scan memory from the location of the last placement and chooses the next available block that is large enough

DYNAMIC PARTITIONING · PLACEMENT

First Fit · Best Fit · Next Fit

One 16 MB request, three policies — each picks a different hole and leaves a different remainder.

First Fit

Scan from the top; take the first hole that fits.

→ takes the 22 MB hole · 6 MB left over

Fastest search. Small leftovers tend to pile up near the start of memory.

Best Fit

Scan every hole; take the smallest one that fits.

→ takes the 18 MB hole · 2 MB left over

Tightest fit per request, yet leaves tiny unusable slivers — often the worst overall.

Next Fit

Like First Fit, but resume from the last allocation.

→ wraps to the 36 MB hole · 20 MB left over

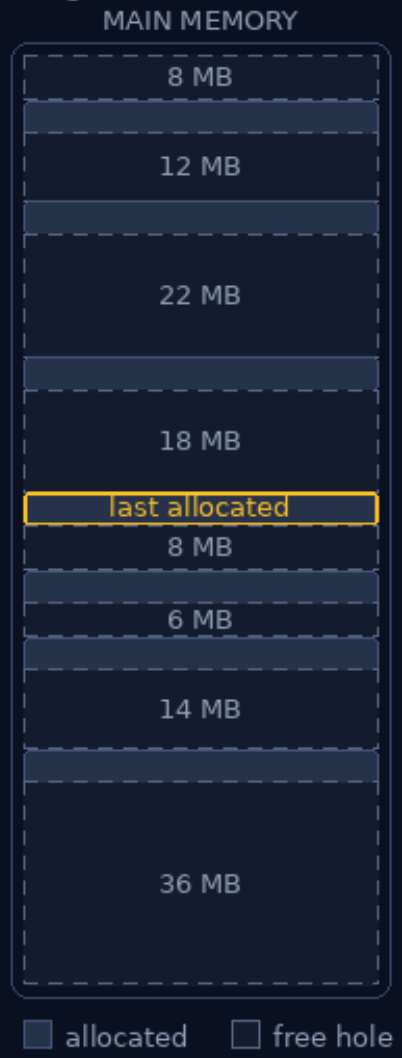
Spreads use across memory; tends to break up the large free block at the end.

Same request, three holes, three remainders — the placement policy is what drives how memory fragments over time.

Placement Algorithms

Allocating a 16 MB block into a fragmented memory

REQUEST 16 MB



First Fit

Scan from the top; take the first hole that fits.

Buddy system

- Combination of fixed and dynamic partitioning schemes.
- Read about this scheme in the book.

Memory deallocation

- When the partitions are not used anymore, they should be released.
- Fixed partitions?
  - Reset the free/busy status of the partition
- Dynamic partitions?
  - Need to combine the free areas.

Question

- Does the schemes that we looked at provide a way to overcome problems arising from relocation requirement?

Relocation — Fixed vs. Dynamic Partitioning

- A process will not always occupy the same place in memory over its lifetime.
  - It can be swapped out to disk, then swapped back into a different partition or hole than before.
  - Under dynamic partitioning, compaction can also shift a resident process to merge scattered free space.
- Dynamic partitioning → relocation is **always** required.
  - Swap-in lands the process in a different hole, and compaction moves processes around.
- Fixed partitioning → relocation is required **only if** a process can be reassigned to a different partition.
  - Single queue for unequal partitions: the process may be reloaded elsewhere → relocation needed.
  - Process always returns to its own partition: its load address never changes → no relocation.
- Consequence: the addresses a process references are **not fixed**.
  - They change each time the process is swapped in or shifted.
  - So addresses must be resolved at run time — via base (and bounds) registers.

Types of addresses

Physical or Absolute

- Actual location in main memory

Logical

- Reference to a memory location independent of the current assignment of data to memory

Relative

- A particular example of logical address, in which the address is expressed as a location relative to some known point

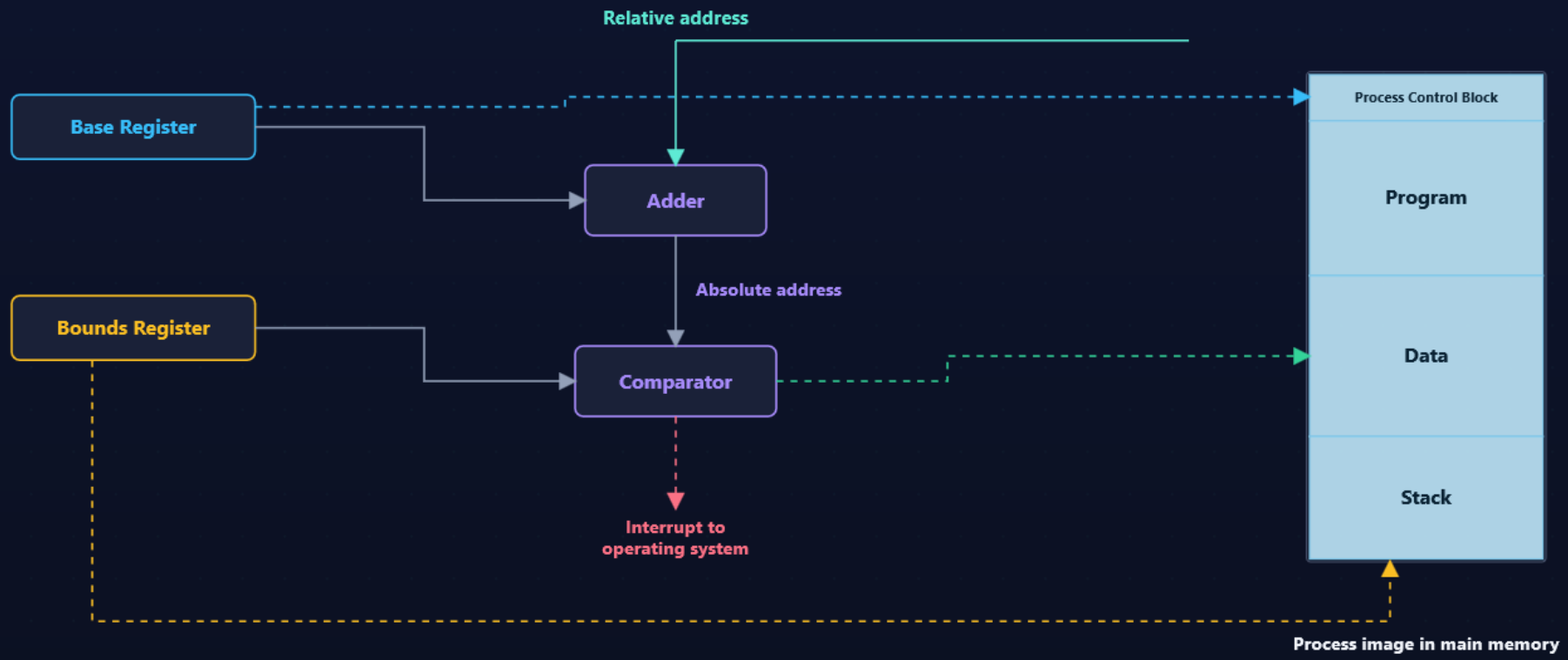
Relative Addressing

- Memory reference in a loaded process are relative to the origin of the program.
- Hardware mechanism to translate relative addresses to physical addresses.
- Bounds register: ending location of the program
- Base register: starting address of the running program

RELOCATION · BASE + BOUNDS

Hardware Support for Relocation

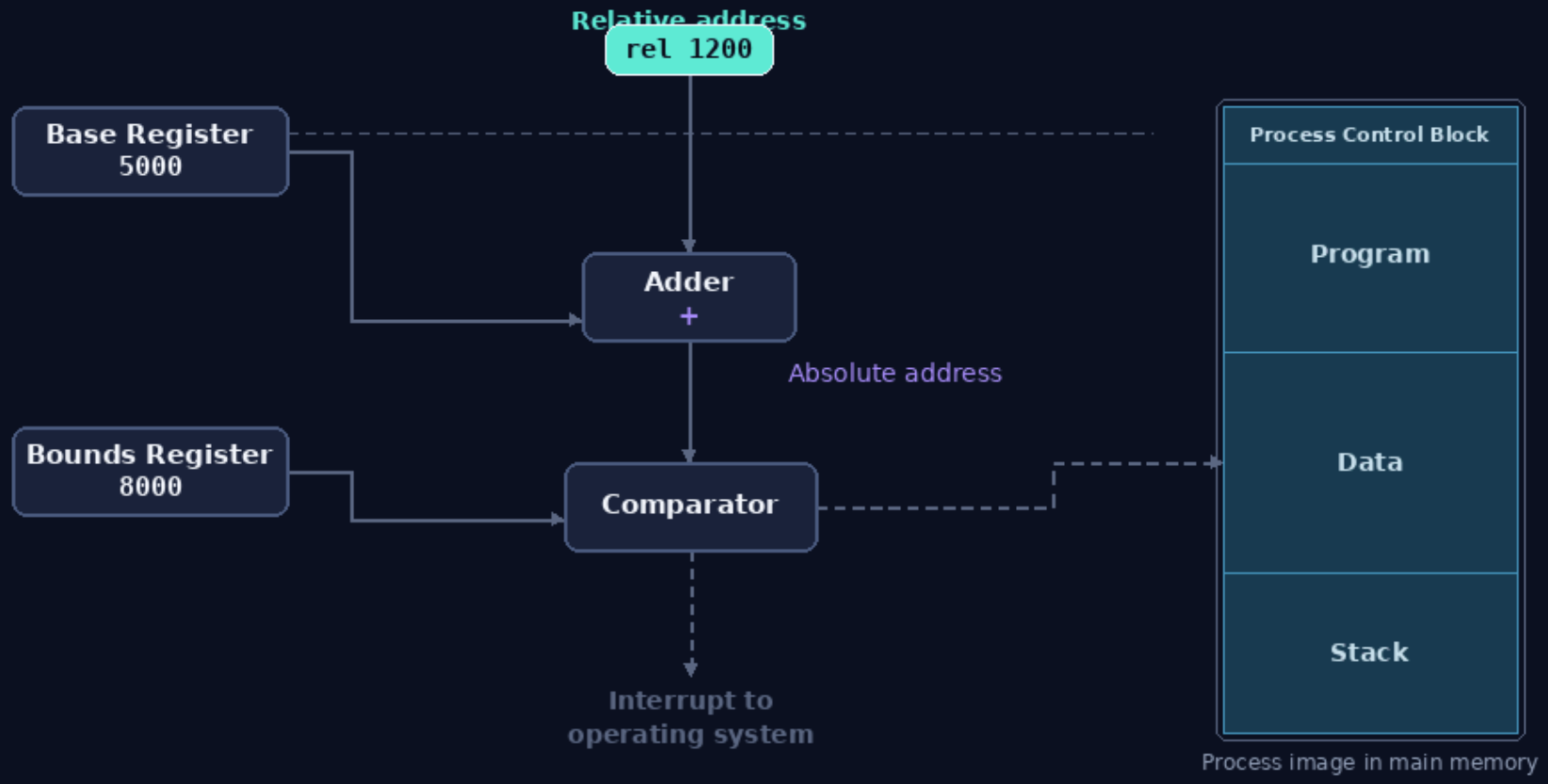
Every memory reference: add the base to relocate it, then check the bound to protect it.



- Base register → relocation (start address of the process image)
- Bounds register → protection (end address; out-of-range traps to the OS)

Hardware Support for Relocation

Every memory reference: the base register relocates it, the bounds register protects it.



① Relocate: absolute = relative + base

② Protect: is absolute within bounds?

Memory management terms

Frame	Fixed-length block of memory
Page	A fixed-length block of data that resides in secondary memory (such as a disk). A page of data may temporarily be copied into a frame of main memory.
Segment	A variable-length block of data that resides in secondary memory. An entire segment may temporarily be copied into an available region of main memory (segmentation) or the segment may be divided into pages which can be individually copied into main memory (combined segmentation and paging).

Frame · Page · Segment

1 / 3 Frame & Page

Fixed vs. variable blocks, and how each moves from disk into main memory.

Frame

a fixed-length block of main memory (an empty slot).

Paging

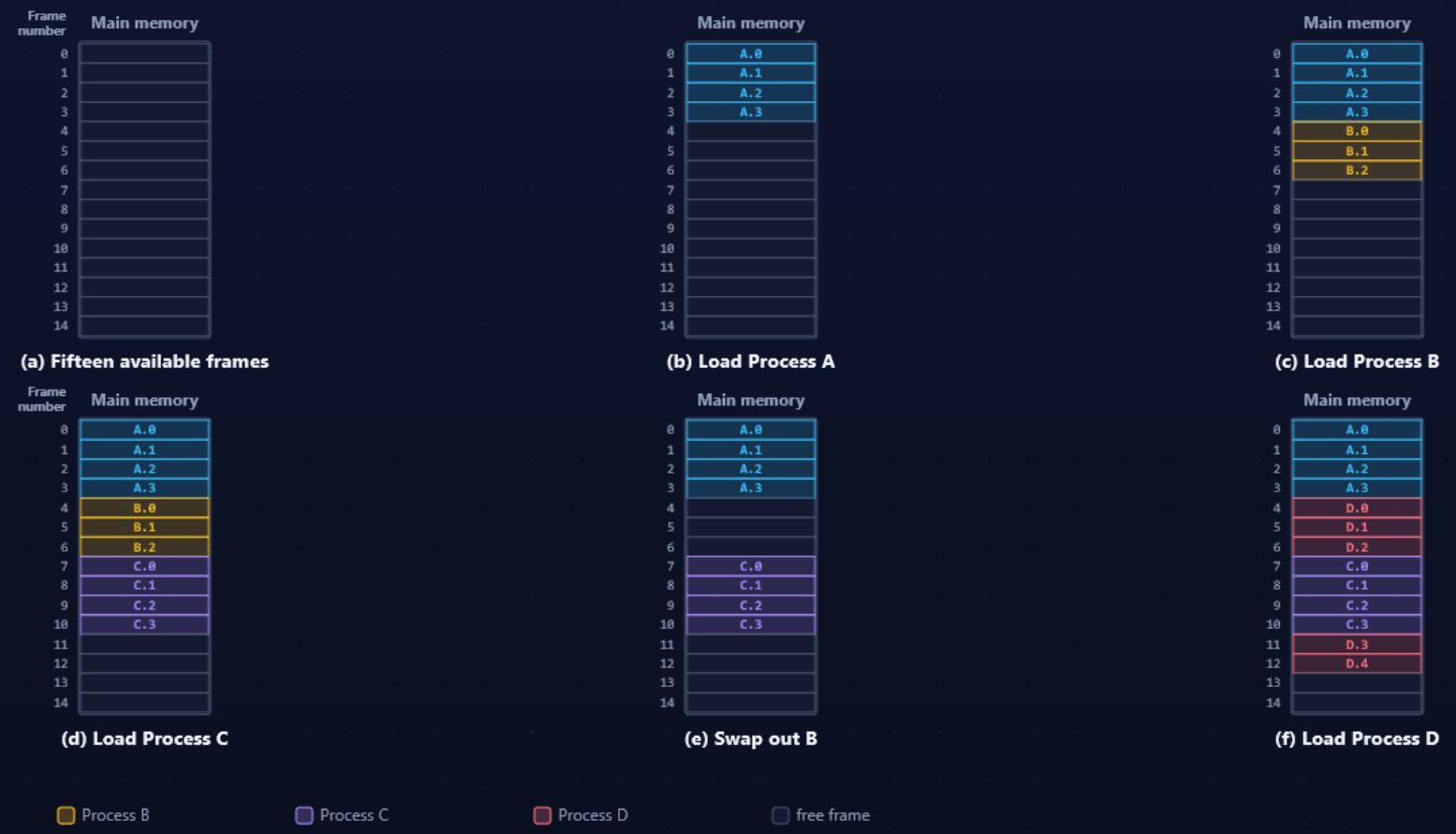
- Partition memory into equal fixed-size chunks that are relatively small
- Process is also divided into small fixed-size chunks of the same size

Pages	Frames
• Chunks of a process	• Available chunks of memory

PAGING · PAGE-TO-FRAME ASSIGNMENT

Assignment of Process Pages to Free Frames

Six snapshots: loading and swapping map each page into any free frame — not necessarily contiguous.



Assignment of Pages to Free Frames

One process page fills one free frame — the frames need not be contiguous.

a · 15 free frames

Frame	Main memory
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

OS FREE-FRAME LIST

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14

Free frames

The OS keeps a list of free frames. Here all 15 are free.

Page Table

- Maintained by the OS for each process.
- Contains the frame location for each page in the process.
- Processor should have a mechanism to access the page table for the current process.
- Used by the processor to produce the physical address.
- Page size is usually a power of 2.

PAGING · DATA STRUCTURES

Data Structures at Epoch (f)

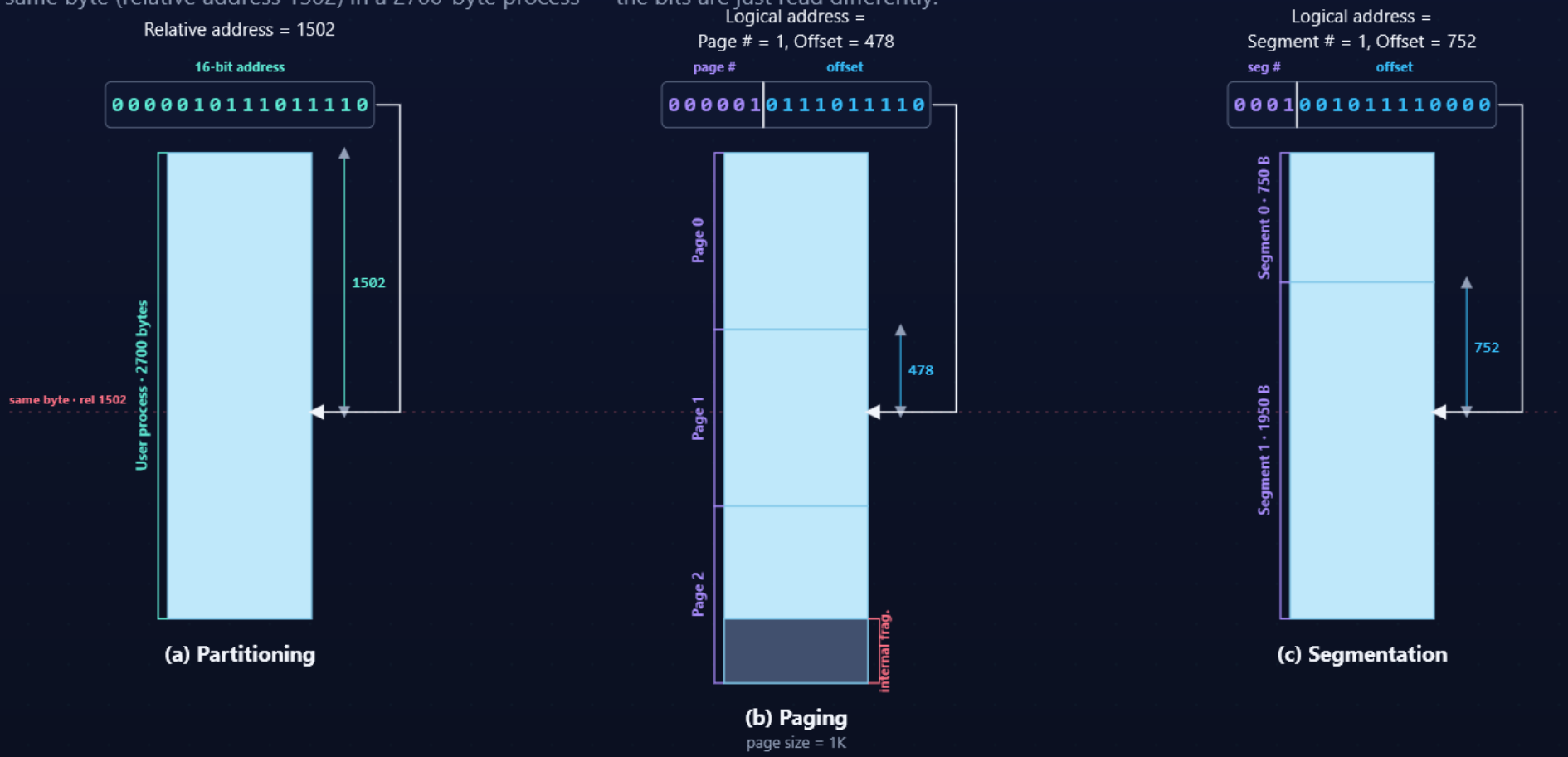
A page table per process (page → frame), plus the OS list of frames still free.

Data structures for the paging example at time epoch (f).

PARTITIONING · PAGING · SEGMENTATION

Logical Addresses

The same byte (relative address 1502) in a 2700-byte process — the bits are just read differently.

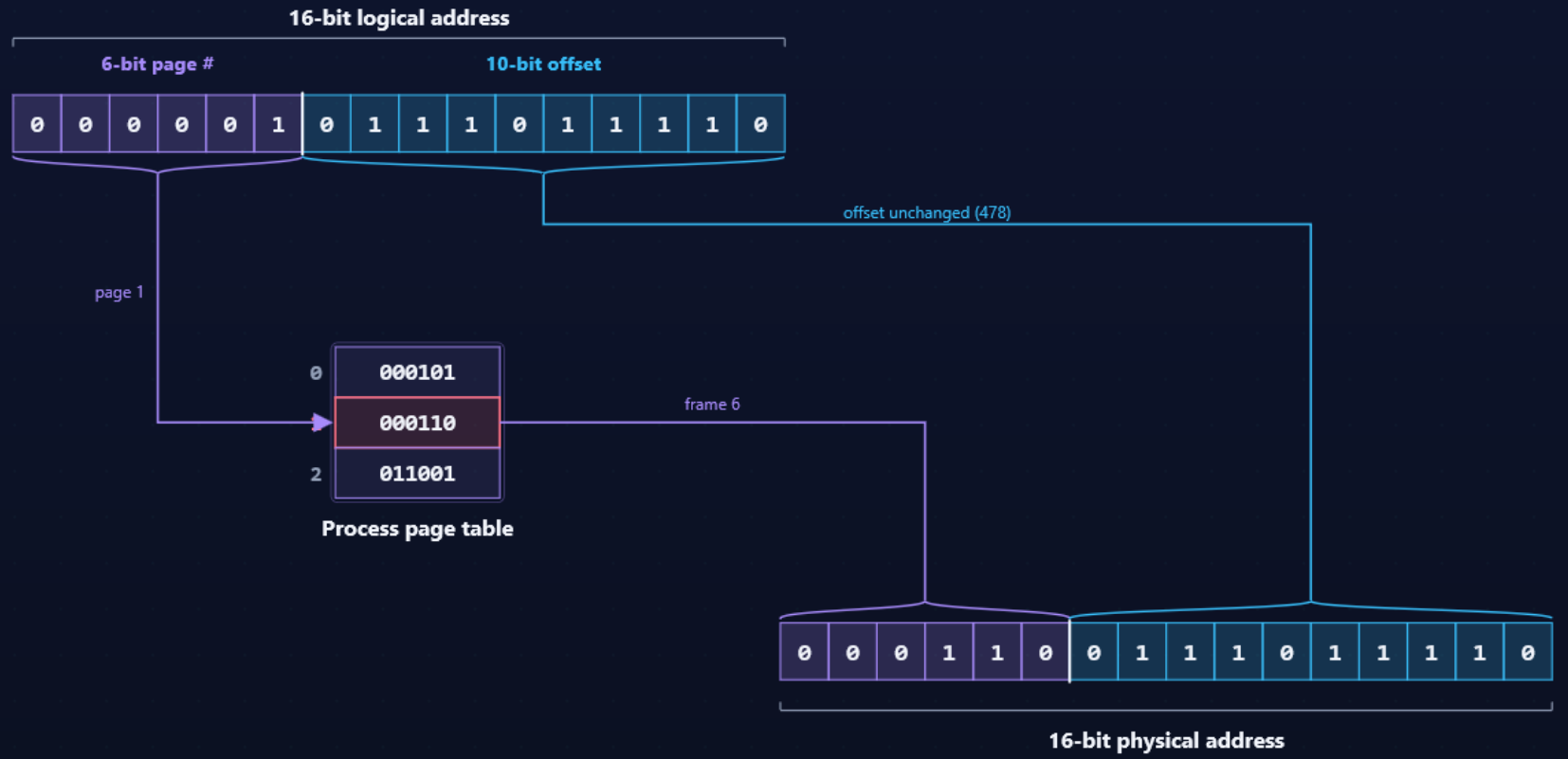


Partitioning: a flat address. Paging: fixed split → page # + offset (note the wasted tail). Segmentation: variable split → segment # + offset.

PAGING · LOGICAL → PHYSICAL

Address Translation

The page number indexes the page table for a frame number; the offset passes through unchanged.



(a) Paging — page # 1 becomes frame # 6 via the page table; the 10-bit offset (478) is copied straight through.

Segmentation

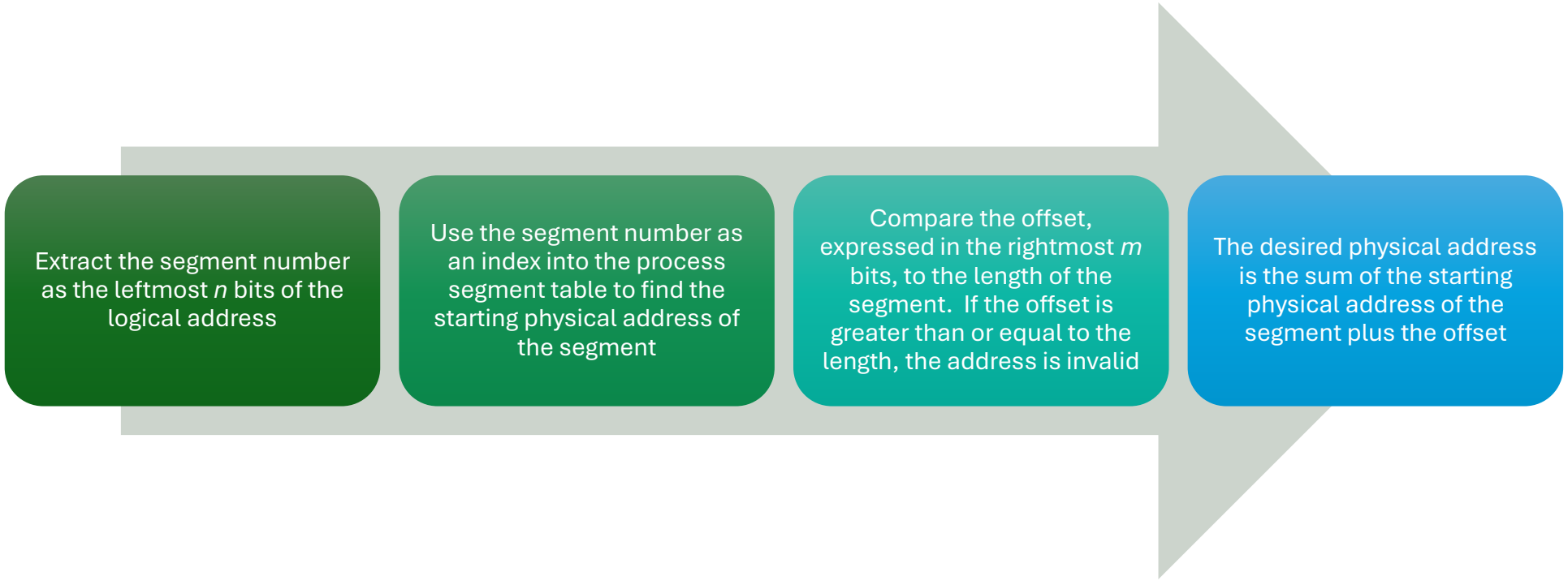
- A program can be subdivided into segments
  - May vary in length
  - There is a maximum length
- Segment table to represent the starting location and size of each segment loaded into the main memory.
- Addressing consists of two parts:
  - Segment number
  - An offset
- Similar to dynamic partitioning
- Eliminates internal fragmentation

Segmentation

- Usually visible to the programmer.
- Provided as a convenience for organizing programs and data
- Typically, the programmer/compiler will assign programs and data to different segments
- For purposes of modular programming the program or data may be further broken down into multiple segments
    - The principal inconvenience of this service is that the programmer must be aware of the maximum segment size limitation

Address translation

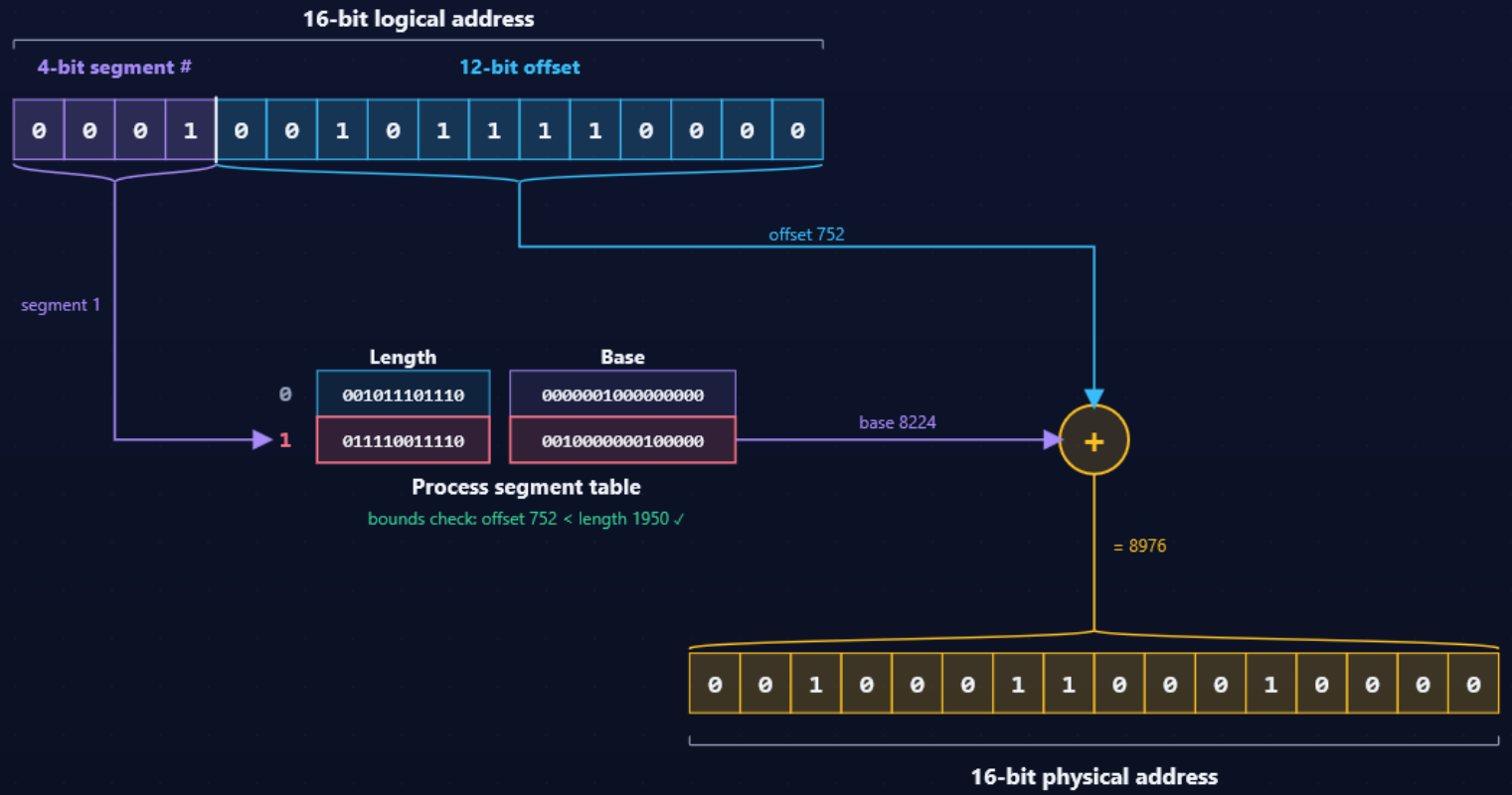
- Another consequence of unequal size segments is that there is no simple relationship between logical addresses and physical addresses
- The following steps are needed for address translation:



SEGMENTATION · LOGICAL → PHYSICAL

Address Translation

The segment number selects a base address, which is **added** to the offset (after a length/bounds check).



(b) Segmentation — segment # 1 selects base 8224; physical = base + offset = 8224 + 752 = 8976. The offset is added, not copied.